

Performance Testing

Date	06 July 2026
Team ID	
Project Name	Credit Card Approval Prediction System
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how the Credit Card Approval Prediction system behaves under expected and peak load conditions. The sections below document the testing approach, tool used, and the results actually obtained by running the live application.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Custom Python load-test script (concurrent.futures.ThreadPoolExecutor + urllib) driving repeated POST requests
Type of Testing	Baseline, Load, Stress, and Spike Testing
Target Module / API	POST /predict — form parsing, feature-vector construction, and XGBoost inference
Test Environment	Local (Flask development server, single process, single-threaded — app.run() default)
Test Date	06 July 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Single-user baseline — sequential requests to establish a floor for response time	1	10	Response time reflects pure model inference latency with no queueing
2	Normal load — sustained concurrent traffic	10	15	Response time stays low and stable; no errors

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
3	Peak / stress load — heavier concurrent traffic	50	15	System stays responsive; error rate remains 0%; latency may rise
4	Spike test — sudden burst of concurrent users	100	10	Server does not crash or error out; degrades gracefully under queueing

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status	Remarks
1	Response Time (Avg)	< 2 seconds	0.66 s (worst case, 100 users)	Pass	Avg rose from 6 ms (1 user) to 663 ms (100 users) as requests queued
2	Response Time (Max)	< 5 seconds	0.76 s (worst case, 100 users)	Pass	Max latency observed under the 100-user spike test
3	Throughput (Req/sec)	—	~156.85 rps (1 user) plateauing to ~146.07 rps (100 users)	Informational	Throughput plateaus quickly — see bottleneck note below
4	Error Rate	< 1%	0%	Pass	No failed requests across all four scenarios (7,664 total requests)
5	CPU Utilization	< 80%	62.2% (peak, at 100 users)	Pass	CPU never approached saturation despite rising latency
6	Memory Utilization	< 80%	~12.5%	Pass	Memory usage stayed flat and low across all scenarios

Step 4: Observations & Analysis

Key Findings:

Response times stayed well under both the 2-second average and 5-second maximum targets even at 100 concurrent virtual users (worst-case average 0.66 s, max 0.76 s). Error rate was 0% throughout all 7,664 requests issued across the four scenarios. However, throughput plateaued almost immediately at roughly 150 requests/second regardless of how many virtual users were added (1 to 100), and average response

time grew roughly 100x (6 ms $\rightarrow$ 663 ms) over that same range.

Bottlenecks Identified:

The Flask app is run via the built-in development server (`app.run()`), which is single-threaded/synchronous by default. Concurrent requests are handled one at a time and queue behind each other rather than executing in parallel — this is why response time grows almost linearly with concurrent users while CPU utilization stays modest (never above ~62%): the bottleneck is request concurrency handling, not raw compute cost of the XGBoost inference itself.

Optimization Steps Taken:

For this testing round no code changes were made; the results above characterize the current single-threaded configuration. Recommended next steps: run the app behind a production WSGI server (e.g. gunicorn or waitress) with multiple worker processes/threads, or pass `threaded=True` to `app.run()`, to allow the model-inference workload to be served in parallel and raise the throughput ceiling beyond ~150 rps.

Step 5: Screenshots / Evidence

Raw scenario-by-scenario measurements captured by the load-test script:

Scenario	Virtual Users	Requests Sent	Avg (ms)	P95 (ms)	Max (ms)	Throughput (rps)	Errors (%)	CPU (%)
Single user baseline	1	1569	6.2	7.4	31.2	156.85	0	62.2
Normal load	10	2295	65	85.6	141.6	152.58	0	51.4
Peak / stress load	50	2249	335.9	369.3	403.1	147.16	0	53.3
Spike test	100	1551	663.2	717.4	764.8	146.07	0	54